Exercise 8-2: Analyze the Cars data

Did you use a language model?

A : No, I did not find any use for a language model for this assignment except for writing comments.

Import the data

In [1]:
```
import pandas as pd
import seaborn as sns
```

In [2]:
```
# Read a pickle file containing data on cars
cars = pd.read_pickle('cars.pkl')
```

In [3]:
```
# Display the first few rows of the cars DataFrame
cars.head()
```

Out[3]:

	aspiration	carbody	enginesize	curbweight	price
0	std	convertible	130	2548	13495.0
1	std	convertible	130	2548	16500.0
2	std	hatchback	152	2823	16500.0
3	std	sedan	109	2337	13950.0
4	std	sedan	136	2824	17450.0

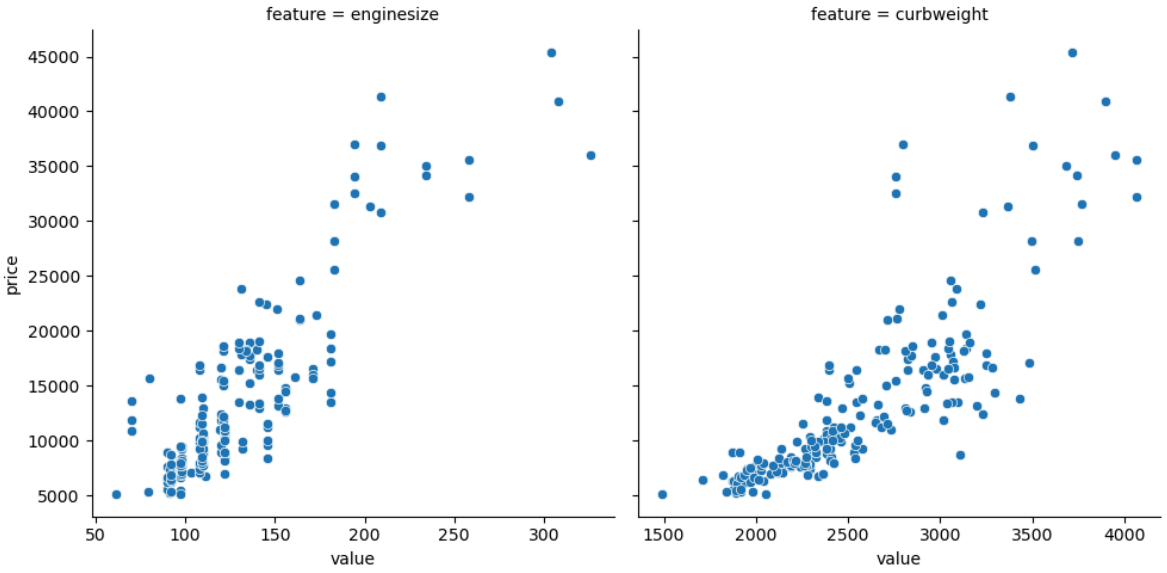
Melt the data

In [4]:
```
# Melt the cars DataFrame to reshape it, keeping 'aspiration', 'carbody', and 'p
# and 'enginesize' and 'curbweight' as value variables, and name the variable co
cars_melt = cars.melt(id_vars=['aspiration','carbody','price'],
                      value_vars=['enginesize','curbweight'],
                      var_name='feature')
```

In [5]:
```
# Create a relational plot using seaborn to visualize the relationship between '
# with separate columns for each 'feature', ensuring independent x-axes for each
sns.relplot(data=cars_melt, x='value', y='price',
            col='feature', facet_kws={'sharex':False})
```

```
C:\Users\Dell\ANACONDA\Lib\site-packages\seaborn\axisgrid.py:118: UserWarning: Th
e figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
```

Out[5]:
```
<seaborn.axisgrid.FacetGrid at 0x1820535a650>
```



Rank the data by price

In [6]:
```
# Create a new column 'priceRank' in cars DataFrame containing the rank of price
cars['priceRank'] = cars.price.rank()
```

In [7]:
```
# Sort the cars DataFrame by 'price' and display the first 10 rows
cars.sort_values('price').head(10)
```

Out[7]:

	aspiration	carbody	enginesize	curbweight	price	priceRank
138	std	hatchback	97	2050	5118.0	1.0
18	std	hatchback	61	1488	5151.0	2.0
50	std	hatchback	91	1890	5195.0	3.0
150	std	hatchback	92	1985	5348.0	4.0
76	std	hatchback	92	1918	5389.0	5.0
32	std	hatchback	79	1837	5399.0	6.0
89	std	sedan	97	1889	5499.0	7.0
118	std	hatchback	90	1918	5572.0	8.5
21	std	hatchback	90	1876	5572.0	8.5
51	std	hatchback	91	1900	6095.0	10.0

Bin the data with quantiles

In [8]:
```
# Create a new column 'priceGrade' in cars DataFrame based on quantiles of 'pric
cars['priceGrade'] = pd.qcut(cars.price, q=3, labels=['low','medium','high'])
```

In [9]:
```
# Display the count of each category in the 'priceGrade' column
cars.priceGrade.value_counts()
```

Out[9]:
```
priceGrade
low       69
medium    68
high      68
Name: count, dtype: int64
```

Group and aggregate the data

In [10]:
```
# Group the cars DataFrame by 'priceGrade', then calculate the minimum and maxim
cars.groupby('priceGrade').price.agg(['min','max'])
```

```
C:\Users\Dell\AppData\Local\Temp\ipykernel_20772\3394201402.py:2: FutureWarning:
The default of observed=False is deprecated and will be changed to True in a futu
re version of pandas. Pass observed=False to retain current behavior or observed=
True to adopt the future default and silence this warning.
  cars.groupby('priceGrade').price.agg(['min','max'])
```

Out[10]:

priceGrade	min	max
low	5118.0	8449.0
medium	8495.0	13860.0
high	13950.0	45400.0

In [11]:
```
# Group the cars DataFrame by 'carbody' and 'aspiration', then calculate the mea
cars.groupby(['carbody','aspiration']).price.mean()
```

Out[11]:
```
carbody      aspiration
convertible  std           21890.500000
hardtop      std           21356.000000
             turbo         28176.000000
hatchback    std            9699.605263
             turbo         13345.243615
sedan        std           13660.371795
             turbo         17307.833333
wagon        std           10973.600000
             turbo         17965.400000
Name: price, dtype: float64
```

In [12]:
```
# Group the cars DataFrame by 'carbody' and 'aspiration', calculate the mean pri
# and unstack the resulting DataFrame to create a pivot table
cars.groupby(['carbody','aspiration']).price.mean().unstack()
```

Out[12]:

aspiration	std	turbo
carbody		
convertible	21890.500000	NaN
hardtop	21356.000000	28176.000000
hatchback	9699.605263	13345.243615
sedan	13660.371795	17307.833333
wagon	10973.600000	17965.400000

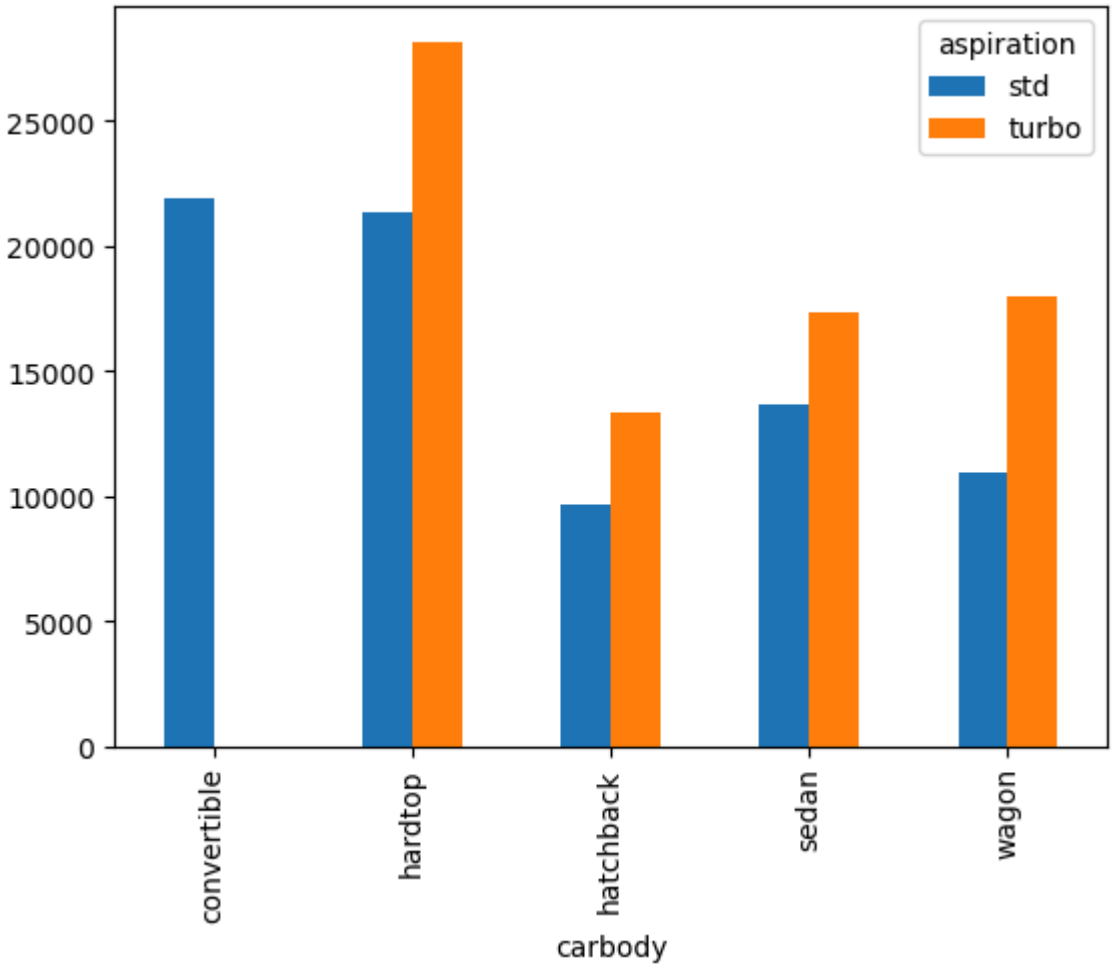
In [13]:
```
# Create a pivot table from the cars DataFrame with 'carbody' as index, 'aspirat
# and 'price' as values, calculating the mean price for each combination of carb
cars.pivot_table(index='carbody', columns='aspiration', values='price', aggfunc=
```

Out[13]:

aspiration	std	turbo
carbody		
convertible	21890.500000	NaN
hardtop	21356.000000	28176.000000
hatchback	9699.605263	13345.243615
sedan	13660.371795	17307.833333
wagon	10973.600000	17965.400000

In [14]:
```
# Create a pivot table from the cars DataFrame with 'carbody' as index, 'aspirat
# and 'price' as values, calculating the mean price for each combination of carb
# then plot the pivot table as a bar chart
cars.pivot_table(index='carbody', columns='aspiration', values='price', aggfunc=
```

Out[14]:
```
<Axes: xlabel='carbody'>
```



In [ ]: